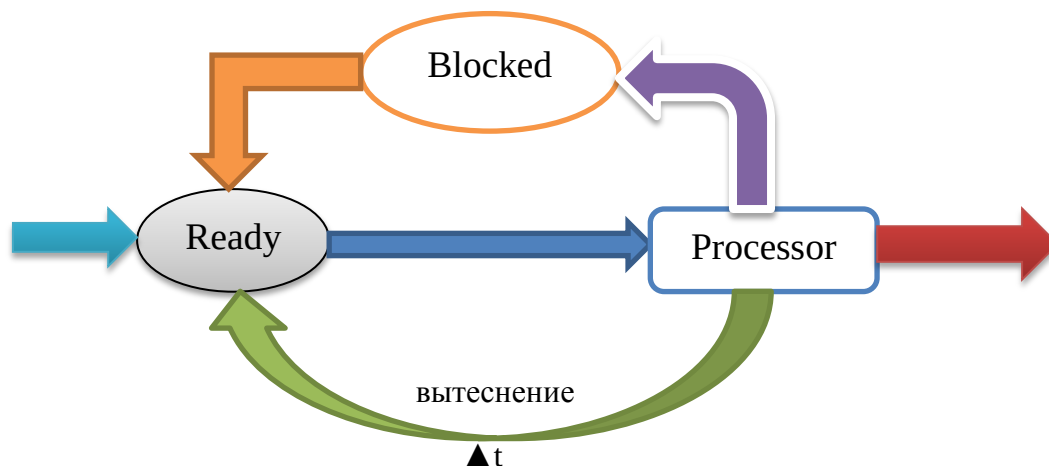


ЛЕКЦИЯ 4. РАСПРЕДЕЛЕНИЕ ВРЕМЕНИ ПРОЦЕССОРА

Главная причина введения принципа мультипрограммирования – стремление компенсировать различие в скоростях ЦП и ВнУ.

Задание пользователя активно, если оно находится в ОЗУ и может выполняться. Совокупность активных заданий – рабочая смесь. Разделяют два режима мультипрограммирования. Состояния процесса: заблокированный, готов, выполняемый.

1. **Разделение времени** – обслуживание задач только в периоды выполнения операций В/В более приоритетными процессами==выполнение задач в фоновом режиме (такой режим планирования используется в системах жесткого РЕАЛЬНОГО времени).
2. **Квантование времени** 1+выделение кванта времени на обслуживание каждого из процессов рабочей смеси в соответствии с некоторой стратегией. Процессу выделяется квант времени и он выполняется до тех пор пока не произойдет одно из условий: а) истечет квант времени, б) процесс заблокирован операциями В/В, в) процесс завершится, г) процесс вытеснен более приоритетным процессом. При этом процесс либо попадает в очередь готовых процессов (а,г), либо попадает в очередь В/В (б), либо выходит из системы (в). Процесс переводится из очереди заблокированных процессов в очередь готовых процессов при снятии блокировки.



Алгоритм планирования очередей готовых процессов включает:

1. Способ выбора готового процесса из очереди готовых процессов **Ready**.
2. Способ вычисления размера кванта времени Δt для выбранного процесса.

Каждому процессу может ставиться в соответствие приоритет (число) означающее важность процесса. Вычисление приоритета основано на статических и динамических характеристиках процесса.

Статические характеристики:

1. Объем занимаемой памяти.
2. Ожидаемое время выполнения.
3. Ожидаемый объем В/В.
4. Коэффициент важности пользователя (внешний приоритет) – влияет на ввод задания в вычислительную систему и перевод в активное состояние.

Динамические характеристики:

1. Ресурсы, находящиеся в распоряжении процесса в данный момент.
2. Общее время ожидания.
3. Количество процессорного времени, полученного в течение предыдущего периода $T \gg \Delta t$ (окно наблюдения).
4. Объем операций В/В в течение предыдущего периода $T_{в/в} \gg T \gg \Delta t$ (окно наблюдения).
5. Общее время нахождения в системе.

При планировании очередей готовых процессов учитываются показатели производительности вычислительной системы:

1. Загрузка ресурсов вычислителя.
2. Времена простоев ресурсов вычислителя.
3. Пропускная способность.
4. Времена ожидания заданий пользователей.

Различают 3 основных класса алгоритмов планирования:

1. По наивысшему приоритету (HPF).
2. Круговорот.
3. Планирование с обратной связью.

ПЛАНИРОВАНИЕ ПО НАИВЫСШЕМУ ПРИОРИТЕТУ:

Если **вытеснение не допускается**, то процесс с наивысшим приоритетом выполняется до тех пор, пока не а) заблокируется, б) завершится. Если в очередь поступает более приоритетный процесс чем текущий, то он должен ждать пока текущий не освободит процессор.

Если **вытеснение разрешено**, то при появлении более приоритетного процесса, текущий процесс прерывается, направляется в очередь готовых процессов и управление передается вновь прибывшему.

ПЛАНИРОВАНИЕ ПО КРУГОВОРОТУ не использует информацию о приоритетах.

Первый процесс из очереди готовых получает квант времени Δt , а затем отправляется в конец очереди, если себя не заблокирует. При **К** процессах каждый процесс выполняется со скоростью, пропорциональной $1/K$. Квант времени при этом необходимо ограничивать снизу временем переключения процессора t ($\Delta t \gg t$).

ПЛАНИРОВАНИЕ С ОБРАТНОЙ СВЯЗЬЮ использует набор из N очередей готовых процессов, каждая из которых обслуживается в порядке поступления (FCFS).

Новый процесс в системе попадает в 1 очередь.

После получения первого кванта и обслуживания на процессоре поступает во 2 очередь.

После получения каждого последующего кванта и обслуживания на процессоре поступает в очередь со следующим номером.

В очереди N процесс обслуживается до завершения.

Кванты времени с ростом номера очереди увеличиваются $\Delta t(i) < \Delta t(i+1)$, $i=1, \dots, N-1$.

Время процессора планируется так, чтобы он **обслуживал непустую очередь с наименьшим номером**.

Планирование с обратной связью обеспечивает лучшее, чем при круговороте обслуживание коротких процессов и не требует никакой предварительной информации (трудоемкие процессы быстро проваливаются в последнюю очередь и не мешают выполнению коротких).

Практика применения алгоритмов планирования.

1. ОС **Реального времени** и **Встроенные системы** используют планирование по наивысшему приоритету.
2. ОС **Разделения времени** и **Диалоговые системы** используют планирование по наивысшему приоритету и круговорот.
3. ОС **Пакетной обработки** используют планирование с обратной связью.

МНОГОУРОВНЕВОЕ ПЛАНИРОВАНИЕ

При реализации планирования необходимо выполнение набора работ:

1. Работа с очередями.
2. Переключение процессов.
3. Вычисление приоритетов.
4. Текущие измерения характеристик процессов.

Метод многоуровневого планирования подразумевает несколько уровней:

1. Диспетчер
2. Краткосрочный планировщик.
3. Долгосрочный планировщик.

Цель: минимизировать затраты на переключение процессов (часто встречающиеся операции должны быть оптимизированы и иметь низкую трудоемкость, в отличие от редких операций).

Диспетчер – вызывается после обработки прерывания, выбирает процесс из очереди готовых процессов и запускает его.

Краткосрочный планировщик – вызывается, чтобы поставить готовый процесс в очередь готовых процессов (выстраивает очередь готовых процессов для диспетчера).

Долгосрочный планировщик – обработка информации о состоянии процесса, предоставленных ресурсах, вычисление приоритета и размера кванта (готовит управляющую информацию для краткосрочного планировщика).

При такой 3-х уровневой организации издержки времени, связанные с планированием, локализуются на различных уровнях.

УПРАВЛЕНИЕ ПАМЯТЬЮ

Управление памятью – отображение информации в память с помощью 3-х функций:

1. **Именуемая функция (И)** – однозначно отображает данное пользователем имя (**ПИ**) в идентификатор информации (программный адрес – **ПА**), к которой это имя относится (переменные, файлы и др. объекты).
2. **Функция памяти (П)** – отображает идентификаторы (программные адреса) в номера физических ячеек памяти (физические адреса – **ФА**), в которых они находятся.
3. **Функция содержимого/значения (З)** – отображает каждый адрес памяти в значение, которое находится по этому адресу.

Исходная информация для загрузчика, объединяющего модули (совокупность модулей + дополнительные данные):

1. Объектный модуль.
2. Длина модуля.
3. Имена описанные в модуле, на которые могут ссылаться другие модули (ссылки внутрь).
4. Имена не описанные в данном модуле, на которые есть ссылки в данном модуле (ссылки наружу).

Загрузчик, объединяющий модули, реализует объединение объектных модулей, настройку адресов, загрузку объединенной программы в ОЗУ.

Загрузчик составляет таблицу имен, на которые могут быть ссылки из других модулей (ссылки внутрь). В таблице каждому имени ставится в соответствие запись следующего содержания:

- а) Если имя описано в модуле, который уже загружен, то запись содержит адрес этого имени относительно начала объединенного имени.
- б) Если объектный модуль, в котором определено это имя, еще не загружен (адрес неизвестен), то запись содержит ссылку на связанный список адресных полей, в которых в дальнейшем нужно будет поместить адрес, соответствующий данному имени. Когда это имя встретится загрузчику, он просмотрит список и вставит адрес этого имени по всем указателям списка.

Все модули загружаются последовательно в ОЗУ. Относительные адреса сдвигаются на сумму длин уже загруженных модулей. Каждая внешняя ссылка либо разрешается (если модуль уже загружен), либо присоединяется к списку, соответствующему этому имени. После загрузки таблицу можно сохранить для отладки.

Узловые моменты привязки приложения к памяти:

1. Программирование в абсолютных адресах – при написании программы указываются абсолютные адреса (объединение отображений **И** и **П** раз и навсегда).
2. Программирование в символьных обозначениях – компилятор порождает адреса вместо символьных имен.

А) Порождаемые адреса – фиксированные абсолютные адреса (привязка **П** происходит сразу же за **И**).

Б) Порождаемые адреса – допускают настройку и при запуске получают абсолютные значения, не меняющиеся при выполнении программы (привязка к памяти **П** происходит во время загрузки).

В) Порождаемые адреса допускают настройку и получают абсолютные значения при каждом обращении к ним (привязки **П** как таковой нет = динамическое распределение).

Основной вопрос задачи распределения памяти состоит в том как объединенный модуль записать в ОЗУ. Далее исходим из того. Что задания переместимы.

Стратегии распределения памяти.

1. Перекрытие программ в памяти (планируется пользователем).
2. Попеременная загрузка заданий.
3. Сегментная организация программ.

4. Страничная организация памяти.
5. Сочетание сегментной организации программ со страничной организацией памяти.

Перекрывание программ

Условие перекрывания: 2 модуля могут располагаться в одних и тех же ячейках памяти, если: а) модулям не нужно присутствовать в ОЗУ одновременно, б) для сохранения межмодульной коммуникационной информации (обычно через поле общей памяти) принимаются необходимые предосторожности.

Для реализации перекрывания редактор связей порождает специальные команды, вызывающие загрузчик всякий раз, когда происходит обращение к одной из перекрывающихся программ (загрузчик вводит модуль в ОЗУ, если он еще не загружен).

Проблемы:

Программист не всегда может априори задать описание перекрываний (т.к. может не знать нужно ли присутствовать перекрывающимся модулям в памяти одновременно).

Т.к. перекрывания планируются пользователем заранее, а степень загрузки памяти и поведение других программ предвидеть заранее невозможно, то эта стратегия не дает существенного улучшения использования всей памяти.

Попеременная загрузка заданий

ОС осуществляет загрузку/выгрузку заданий целиком после начальной загрузки в процессе выполнения смеси заданий. Стратегия позволяет избирательно загружать задания с целью увеличения загрузки все компонент вычислителя.

Сегментная организация программ

Сегментация программ – прием, при котором адресная структура программы отражает ее содержательное членение. Пространство адресов программы разделяется на отрезки – сегменты различной длины, соответствующие содержательно различным частям программы. Сегмент = процедура, область описания данных. При сегментации АДРЕС состоит из 2-х частей:

1. Имя сегмента **S** (его номер).
 2. Адрес слова внутри сегмента **d** (смещение).
- ⊕ а) Сегменты могут размещаться в несмежных областях памяти.
б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).

Реализация

Каждому заданию (приложению), состоящему из набора сегментов, соответствует ТАБЛИЦА СЕГМЕНТОВ (ТС), в которой для каждого сегмента имеется запись (строка). Адрес ТС хранится в аппаратном РЕГИСТРЕ ТАБЛИЦЫ СЕГМЕНТОВ (РТС) каждого процессора.

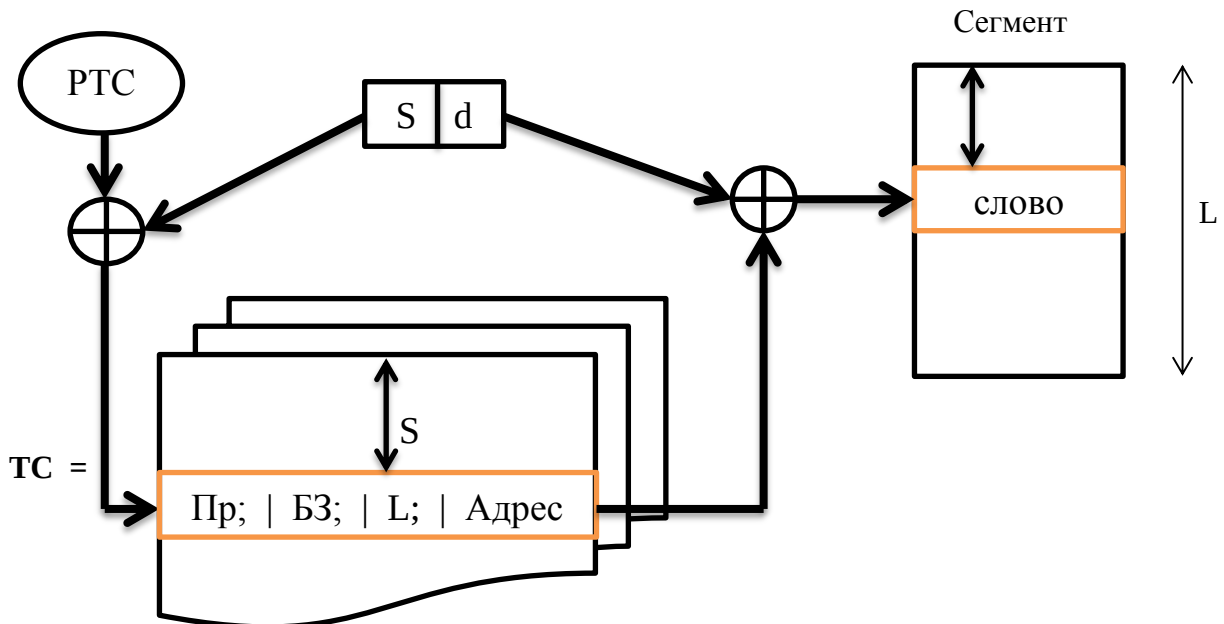
В S-й строке ТС (имя сегмента=№строки) находится:

1. Признак присутствия сегмента в ОЗУ (**Пр**).
2. Адрес (Базовый регистр) S-го сегмента (**Адрес**).
3. Длина (Граница) сегмента (**L**).
4. Биты защиты для контроля доступа к сегменту (**БЗ**).

Доступ к слову [S,d]:

1. По РТС обращение к ТС.
2. Проверяется присутствие сегмента в памяти (по признаку **Пр**)
3. При наличии сегмента в ОЗУ в S-й строке выбирается **Адрес** сегмента.
4. В d-й ячейке выбирается искомое слово.

Если сегмента в ОЗУ нет – происходит прерывание из-за отсутствия сегмента в памяти.



Переключение системы на другую программу (приложение) сводится к замене содержимого РТС на адрес другой таблицы.

- $\oplus$
- а) Сегменты могут размещаться в несмежных областях памяти.
 - б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).
 - в) Перекрытия реализуются системой без предварительных указаний программиста. Возможно перекрытие областей отдельных программ. При этом эффективно используется память.
 - г) Упрощается объединение модулей. Операция установления внешних связей сводится к просмотру таблицы для отыскания адреса (S,d), соответствующего внешней ссылке (не нужны эти функции в редакторе и загрузчике).
 - д) сегментация облегчает организацию параллельно используемых программ. Для этого информацию о параллельно используемой программе вносят несколько таблиц сегментов. Сама программа при этом обрабатывает разные сегменты данных, описанные в таблицах различных приложений.



А) Внешняя фрагментация (пустоты между загруженными в память сегментами, размеры которых малы для размещения других сегментов – образуются из-за загрузки-выгрузки сегментов в процессе вычислений). Профилактика: страничная организация памяти!

Б) Два обращения к ОЗУ (первое – к таблице за адресом сегмента, второе – к адресуемому объекту внутри сегмента). Существенно снижает быстродействие вычислителя. Профилактика: полностью ассоциативное кэширование адресных ссылок на

сегменты (емкость кэша адресных ссылок TLB обычно несколько сотен строк), реализуется с помощью отображения:

Вход=ID-задачи+S (логический адрес)→ Физический адрес сегмента=**Выход**

